Seminar · CS6013

Semantic caching for LLMs

GPT Semantic Cache → ContextCache → SmartCache

Why is Semantic Caching useful?

Every LLM call costs money and latency, yet in FAQ-style traffic many queries are the same question reworded.

Three users ask:

- What is PyTorch?
- Tell me about PyTorch
- Can you explain what PyTorch is?



Exact-match cache

Key: the query text

3 LLM calls

Every rewording is a **new key**, so every query **misses**.

Semantic cache

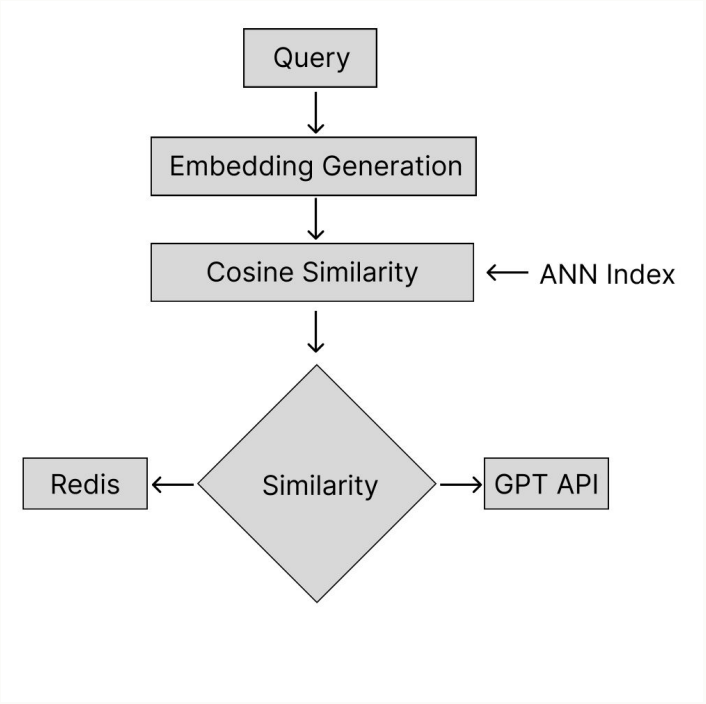
Key: the query's meaning

1 LLM call

Rewordings land on the **same entry** and **reuse its answer**.

The question: can we match a new query to past ones by meaning, and serve the stored answer without calling the LLM?

Semantic GPT Cache



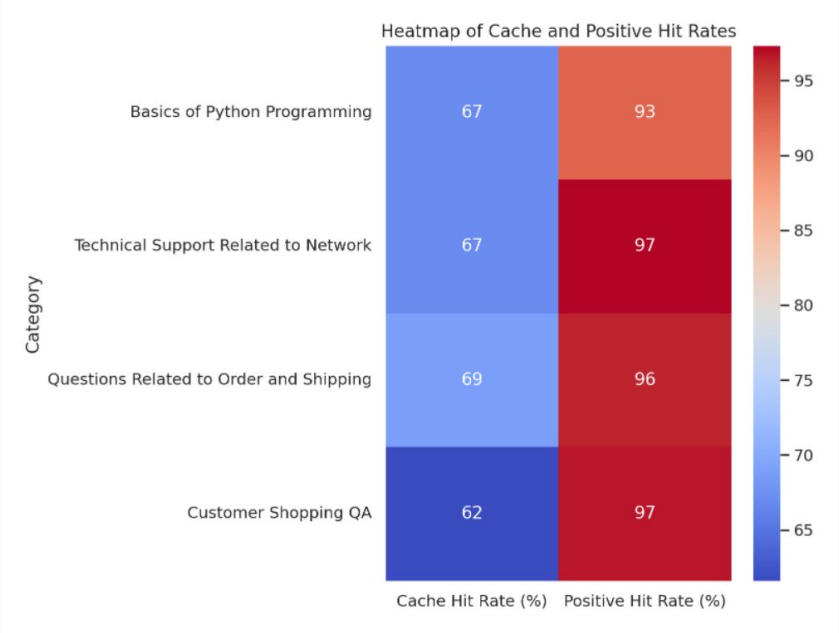
$$\text{hit} \iff \text{sim}(q, c) = \frac{\mathbf{e}_q \cdot \mathbf{e}_c}{\|\mathbf{e}_q\| \, \|\mathbf{e}_c\|} \geq 0.8$$

Each query is turned into a 384-d vector by MiniLM.

An HNSW index finds the nearest cached queries, stored in Redis.

At 0.8 or above the cached answer is served; otherwise GPT answers and the pair is cached.

How does it help?



Data

- **8,000** self-built Q&A pairs in **4 FAQ-style areas**
- **2,000** generated test questions, **500 per area**
- Every test is a **single, stand-alone question**

Result

- **66.1%** of queries hit the cache (**1,322 of 2,000**)
- **95.7%** of hits judged **correct** by GPT-4o-mini

Caveat

- **Narrow, repetitive FAQs** favour high hit rates
- **Real traffic is messier:** open-ended chat is untested

Where a single-query cache breaks

1 One threshold for all

- A single **0.8 cut-off** is used for **every topic**.
- About **1 in 25 hits** was still judged **wrong**.

2 No conversation memory

- Each query is matched **on its own**, with **no chat history**.
- A follow-up like “What are **its** key features?” can hit an answer about a **different topic**.

3 All-or-nothing reuse

- Only **near-repeats** of a cached query benefit.
- A **new question gets nothing**, even if most of its **prompt was seen before**.

ContextCache adds the conversation to the key

A · In the cache

Chat about relational databases

What is a relational database?

What are **its** key features?

Stored answer: features of a relational database

B · Same words, new chat

Chat about cloud computing

What is cloud computing?

What are **its** key features?

GPTCache ✗ exact match, serves the database answer

C · New words, same context

Reworded question

Explain the important features as a relational database

The fix: key = **query + conversation**. Same words in a different chat should **miss**; different words in the same context should **hit**.

Two stages: fast filter, then context check

Stage 1 · Query only

Cheap vector search keeps a shortlist.

$$C_1 = \{\, c \;:\; \cos(\mathbf{v}_Q, \mathbf{v}_c) > \theta_1 \,\}$$

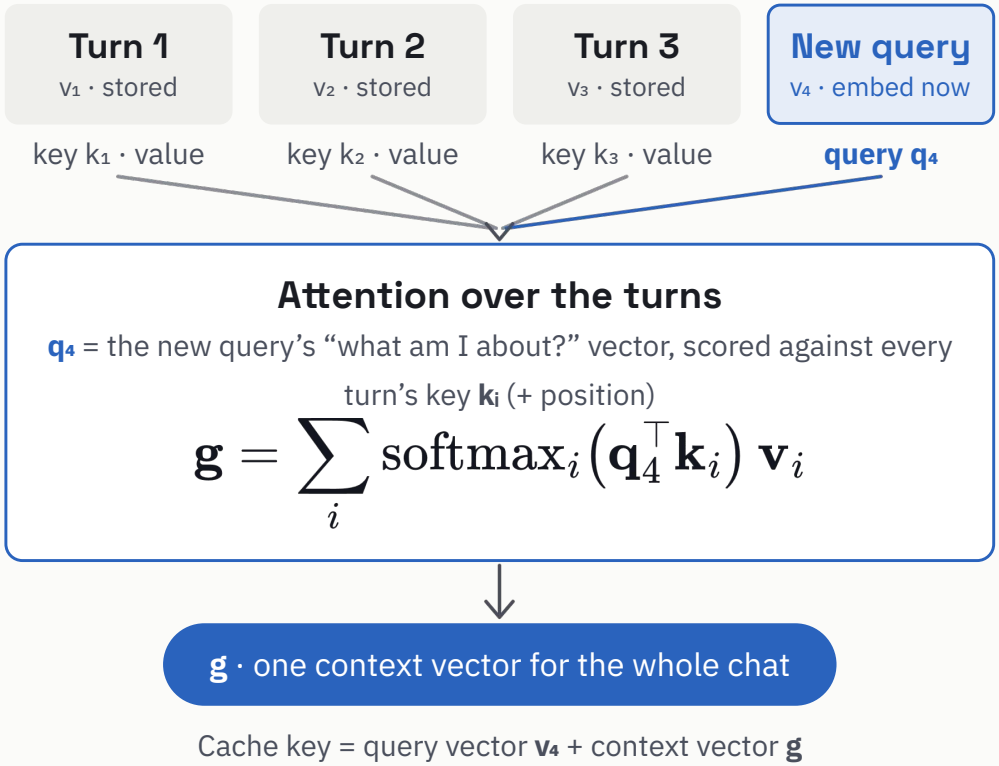
Stage 2 · Query + history

Compare whole conversations, not just queries.

$$\mathbf{g} = \mathrm{SelfAttn}\left(\{\mathbf{v}_Q\} \cup V_H\right)$$

$$c^* = \arg\max_{c \in C_1} \cos(\mathbf{g}, \mathbf{g}_c), \quad \text{hit if } \cos(\mathbf{g}, \mathbf{g}_{c^*}) > \theta_2$$

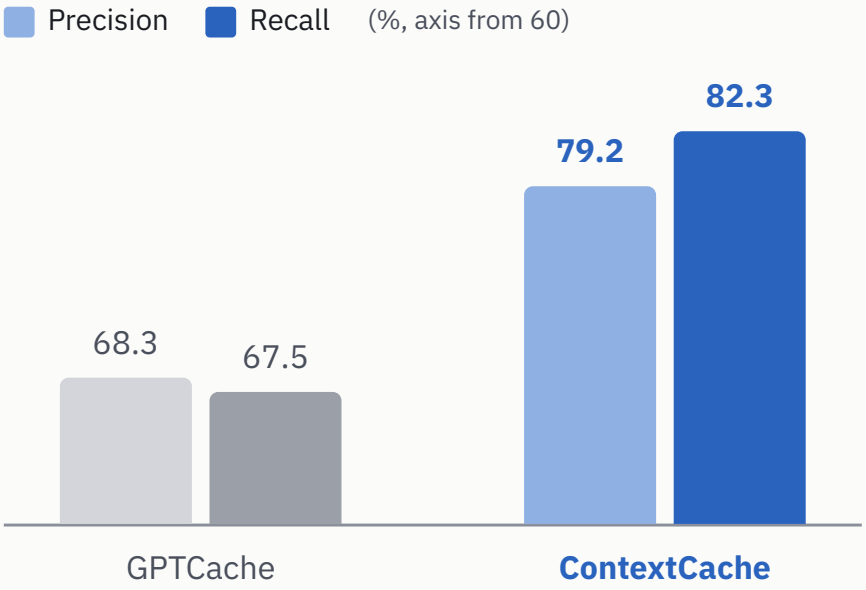
How the conversation becomes a lookup key



Encode each turn once

- Each turn is embedded **once**, when it is asked, and its vector is **stored**
- At lookup, **only the new query** is encoded
- So a **10-turn chat** costs the same to encode as a **1-turn chat because g is always a fixed-size vector.**

More right hits, fewer wrong ones



1,000 ShareGPT queries · 30% of chats cached first · GPT-4 paraphrases as test queries

Fewer wrong hits: precision +10.9 pts

- Targets **look-alikes**: same words, **different chat** (B)
- Stage 2 **rejects** candidates whose **context doesn't match**

More right hits: recall +14.8 pts

- Targets **vague follow-ups**: "What are **its** key features?"
- Context picks the **right cached entry**

Hits stay fast

- A hit is about **10× faster** than calling the LLM

What ContextCache still can't do

1 It caches answers only	Only **final answers** are stored, so a hit needs a **near-repeat** of an earlier question. On a miss, the LLM **reprocesses the whole conversation from scratch**, and nothing computed inside the model is reused.
2 The whole chat becomes one vector	Every turn is compressed into **a single context vector g**. In **long conversations** the history can dominate g and **drown out the new question.**
3 The evaluation is thin	It is tested on **one dataset** of 1,000 ShareGPT queries, using **GPT-4-written paraphrases** rather than real user rewordings. The thresholds **θ_1 and θ_2** and the **training procedure** are not reported.

→ **Next: SmartCache** keeps the context-awareness but moves it **inside the inference engine**, reusing the KV cache too.

Same path, then a detour: pay for everything

User B follows the conversation User A already had, served from an answer cache, then **asks something new**.

Turn 1	Tell me about PyTorch	**HIT**	LLM work: **0**
Turn 2	How can I install it?	**HIT**	LLM work: **0**
Turn 3	Does it support Apple GPUs?	**MISS**	Prefill **whole chat**

Hits return **text only**: they leave **no model state**. So at the detour the LLM must read the **entire conversation** from scratch, even though A's session already computed it.

Prefill at turn 3

Answer cache: System prompts+ history + question



≈ 424

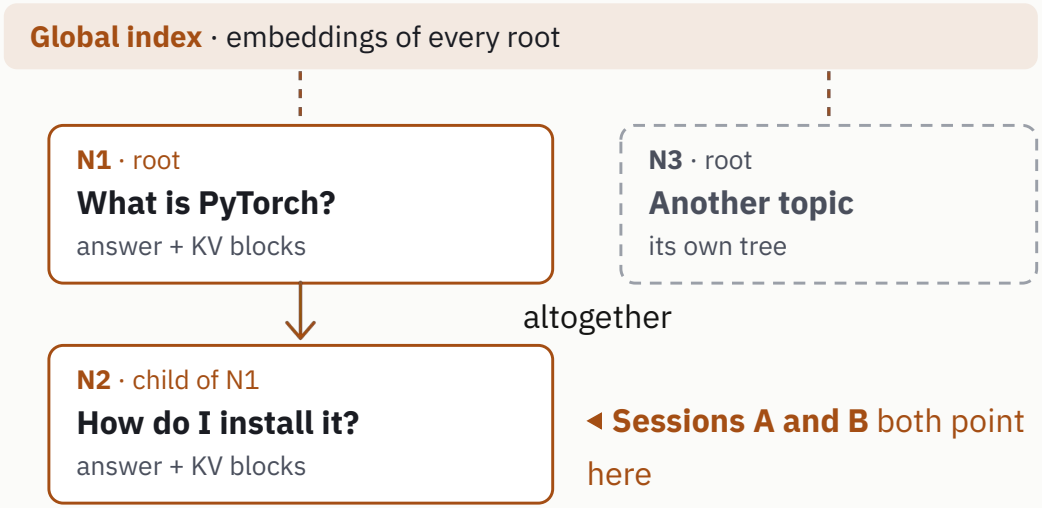
Reusing A's context: question only

≈ 8

Tokens, illustrative. The gap **grows with every turn** the paths share.

Key insight: the KV cache is the model's state for the **context it has already read**. If B's path means the same as A's, let B **share A's context**: its KV is reused exactly, and only the **new question** needs prefill.

How the cache is built: the Semantic Forest



What's stored, and where

Query embedding — CPU RAM
→ parent's child index (or global index)

Cached answer
→ the node itself

Child index — CPU RAM
→ the node (a small FAISS index)

KV cache — GPU memory
→ GPU blocks, via the node's block table

1 · User A
"What is PyTorch?"
Forest is empty → LLM answers → new root **N1**

2 · User B, new session
"Tell me about PyTorch"
Global search over roots → **N1** (≥ 0.75) → **hit**

3 · User A
"How do I install it?"
No child of N1 matches → LLM answers → child **N2**

4 · User B
"How can I install it?"
Search N1's children → **N2** → **hit**; B shares N2's KV

Attention peeking: follow-up or new topic?

Runs only on a **double miss**: the LLM must answer anyway, so SmartCache reads the new question's **attention during prefill** to decide where the new node goes.

The score

$$s = \frac{1}{H\,|Q|} \sum_{h=1}^{H} \sum_{i \in Q} \sum_{j \in C} A_{ij}^{(h)}$$

$A_{ij}^{(h)}$	attention from question token i to token j, in head h
Q	the new question's tokens
C	history tokens, minus the **first 8 sinks**
$\sum_{j \in C} A_{ij}^{(h)}$	share of token i's attention on the history (0–1)
$\frac{1}{H\,\lvert Q\rvert} \sum_{h} \sum_{i \in Q}$	average over H heads and |Q| tokens gives one score **s**

τ is set offline per model from 5 standalone questions; values are not reported.

s ≥ τ → child node

At N1 · How do I install it?

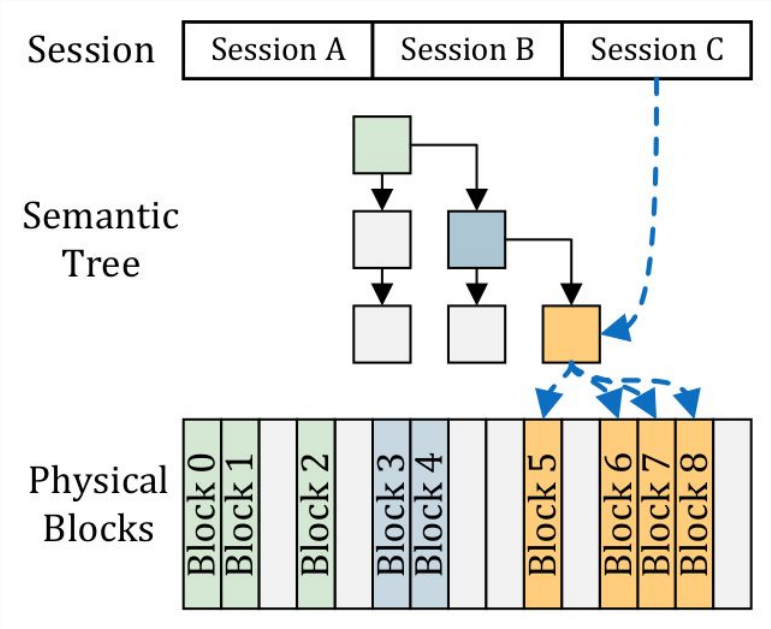
"it" **needs the history**. Decode with it, add a **child** of the current node; **history KV reused**.

s < τ → new root

At N1 · How do I study for Endsems?

Self-contained. Restart **without history**; new **root** in the global index.

One copy of the KV cache, many sessions



Session C's dashed arrow points to the **orange node**, its current position

The orange node's block table maps to **Blocks 5, 6, 7, 8**, not contiguous

Colours match nodes to blocks: **green root → 0–2**, **blue node → 3–4**

C's path is green → blue → orange, so its KV spans **Blocks 0–8**

Another session on the same path **points at the same blocks** instead of computing its own

Why it's exact: a session on a node **has that path's exact tokens** at the same positions, so its shared blocks equal what it would compute. Only the **0.75 match** is approximate.

How to handle a limited KV-cache budget?

STEP (Semantic-aware Tiered Eviction/Prefetching) manages GPU KV memory as **one policy with two halves**, both driven by the semantic forest:

1 · Eviction

Who leaves when GPU memory is full

$$S_{\text{evict}}(N) = \frac{\alpha}{\text{depth}(N)} + \frac{\beta}{\#\,\text{sessions}(N)}$$

Stated intent: evict **deep** nodes and nodes **few sessions share** first

N1 · root, 3 sessions	0.6·1 + 0.4·⅓ = **0.73**
N5 · depth 3, 1 session	0.6·⅓ + 0.4·1 = **0.60**

2 · Prefetching

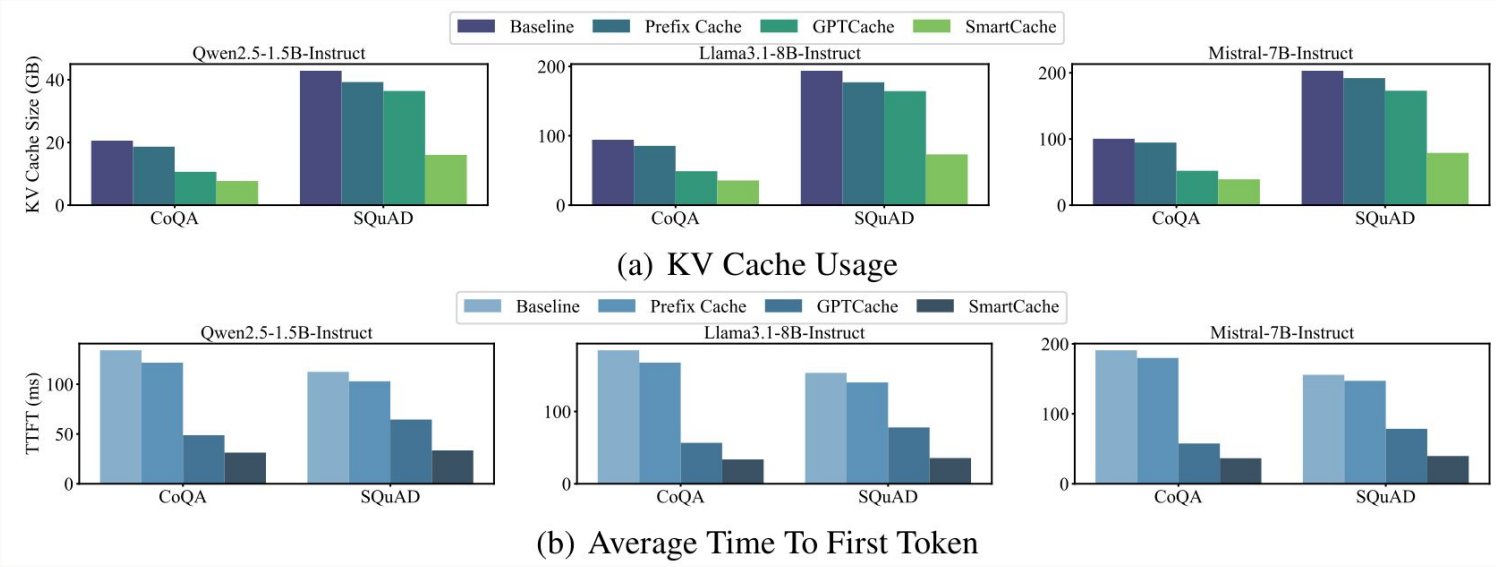
Load the likely next turn early

$$\text{AF}(C_i) = \frac{\#\,\text{sessions } N \to C_i}{\#\,\text{sessions at } N}$$

When a session reaches **N**, its **most popular children** are loaded onto the GPU, counted over a recent time window

At N1: **N2**, 2 of 3 sessions	AF = **0.67**
At N1: **N3**, 1 of 3 sessions	AF = **0.33**

Less memory, faster first token, every model



−59.1%
KV memory vs prefix cache

−78.0%
First token vs prefix cache

−37.7%
First token vs GPTCache

+39.9%
F1 vs GPTCache (Qwen)

Where the gains actually come from

Faster first token

Answer hits skip the model entirely

When a question matches a node, its **stored answer streams straight from the cache**: no prefill and no decoding. The first token waits only for the **embedding and a vector search**, which is why TTFT drops so sharply.

Less KV memory

Shared nodes keep one copy

Sessions that reach the same node **point at its existing KV blocks** instead of computing their own. Reworded sessions of the same conversation hold **one copy of the history**, where prefix caching would store one per session.

vCache

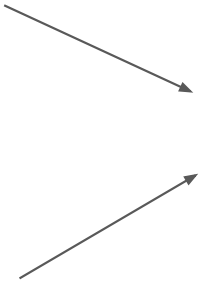
Verified semantic prompt-response caching

Anurag Borkar, 25M2119

Semantic Prompt Caching

"*Explain how PCA works.*"

"*What is Principal Component Analysis?*"

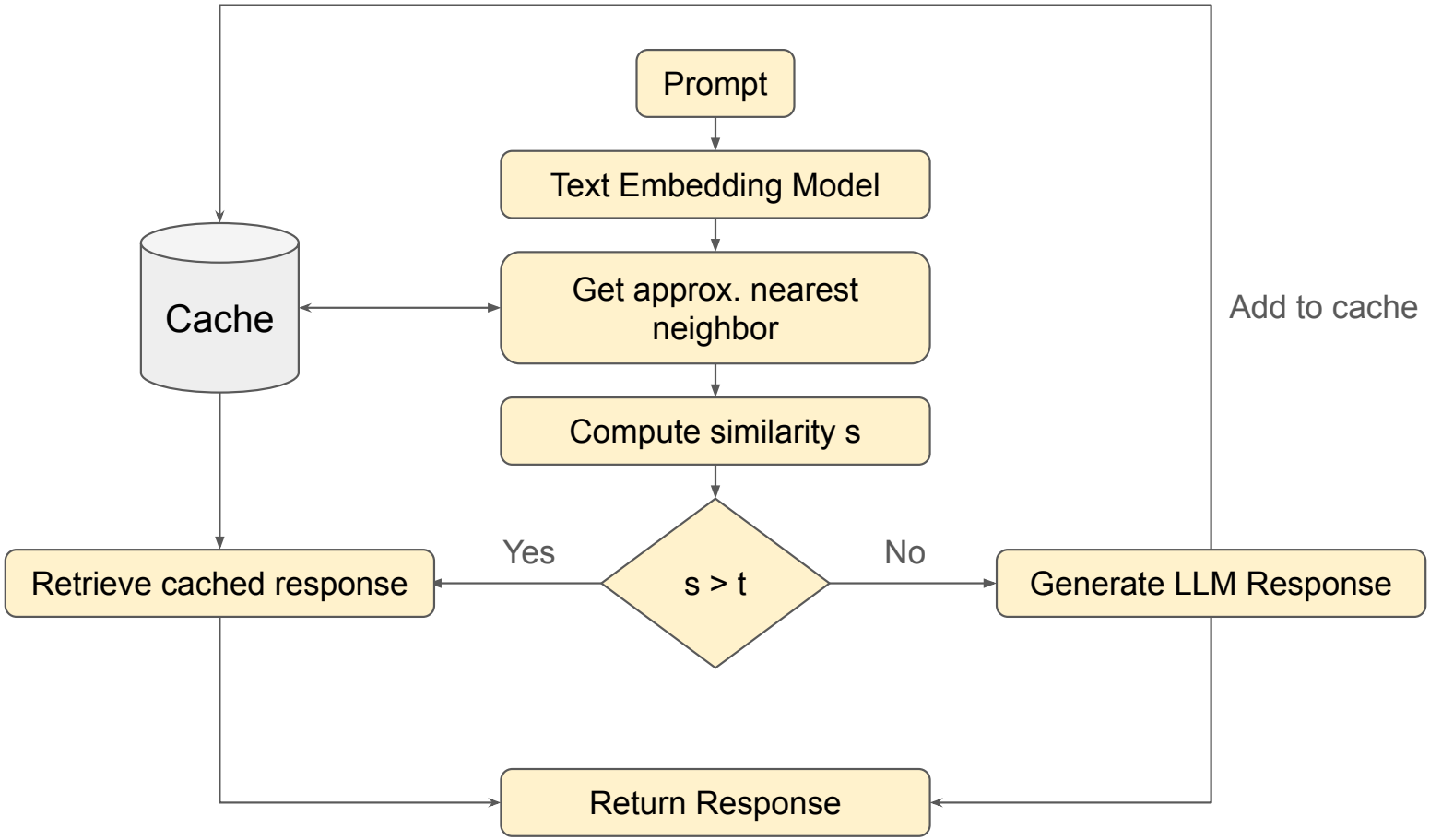


Identical response can work

"*How to code up Principal Component Analysis?*"

Needs different response

Semantic Prompt Caching



Thresholding Dilemma

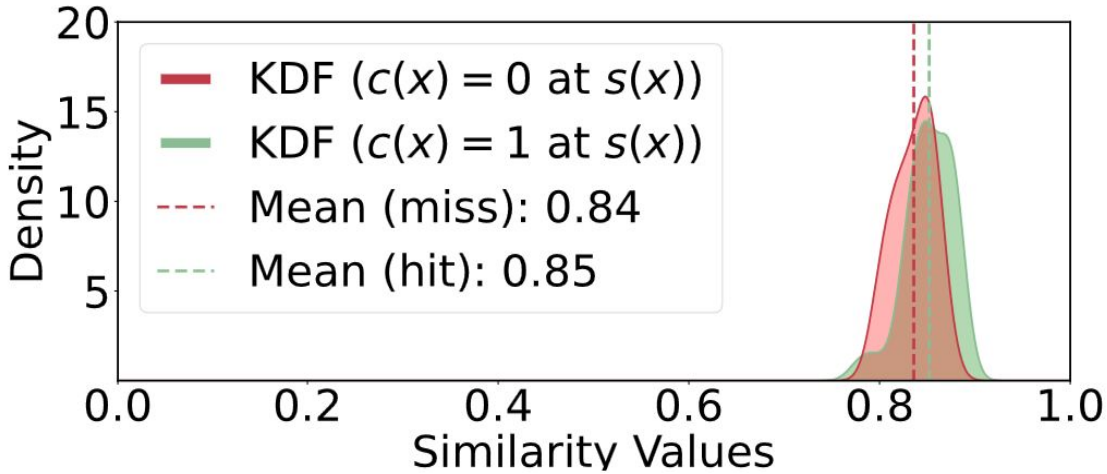


less cache misses
more errors

more cache misses
less errors

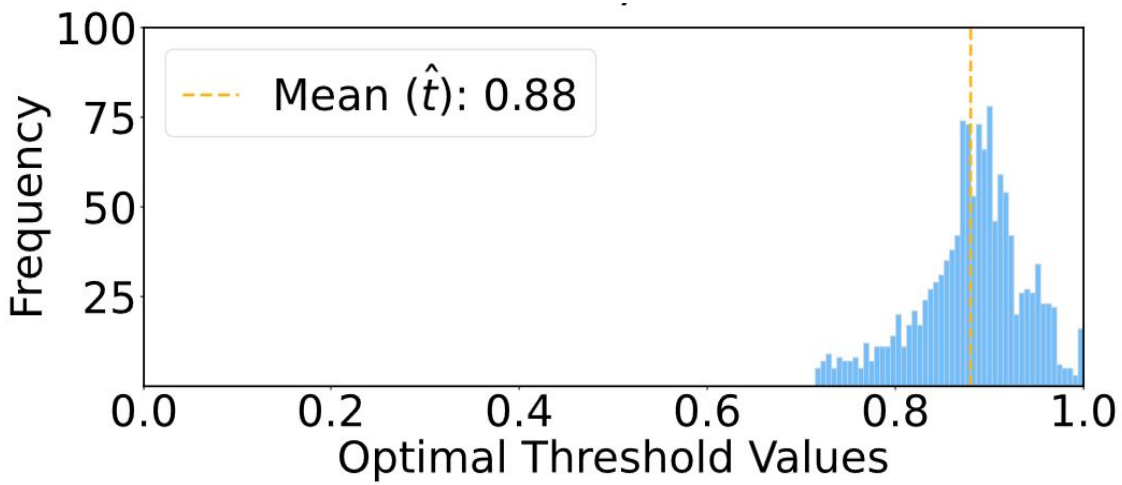
No Error Guarantees!

Empirical Evidence is against single threshold



A single threshold for the full dataset is not possible without sacrificing cache hits

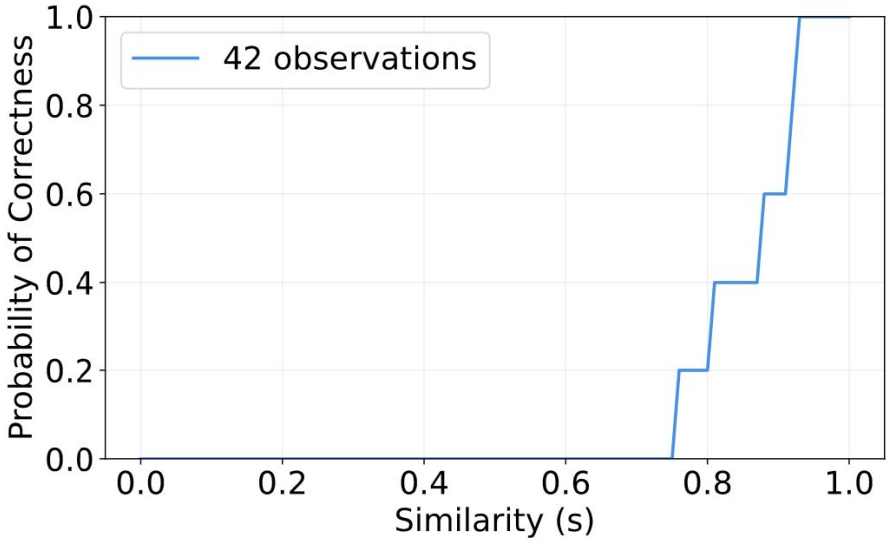
Prompt-wise threshold distribution gives a clue



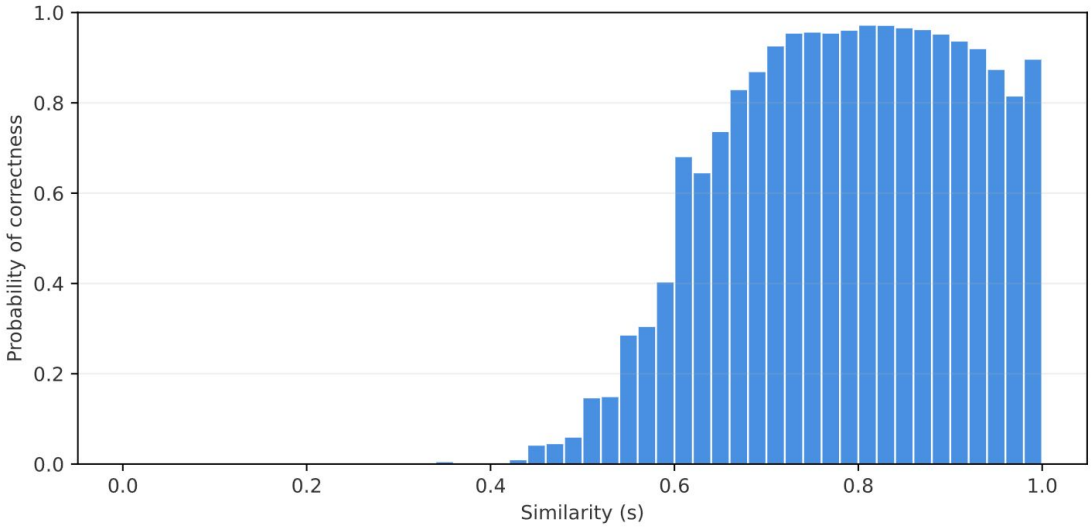
Optimal threshold varies per prompt and can be exploited

If the threshold is prompt-specific, can we estimate the correctness based on similarity?

Correctness as function of similarity



One embedding

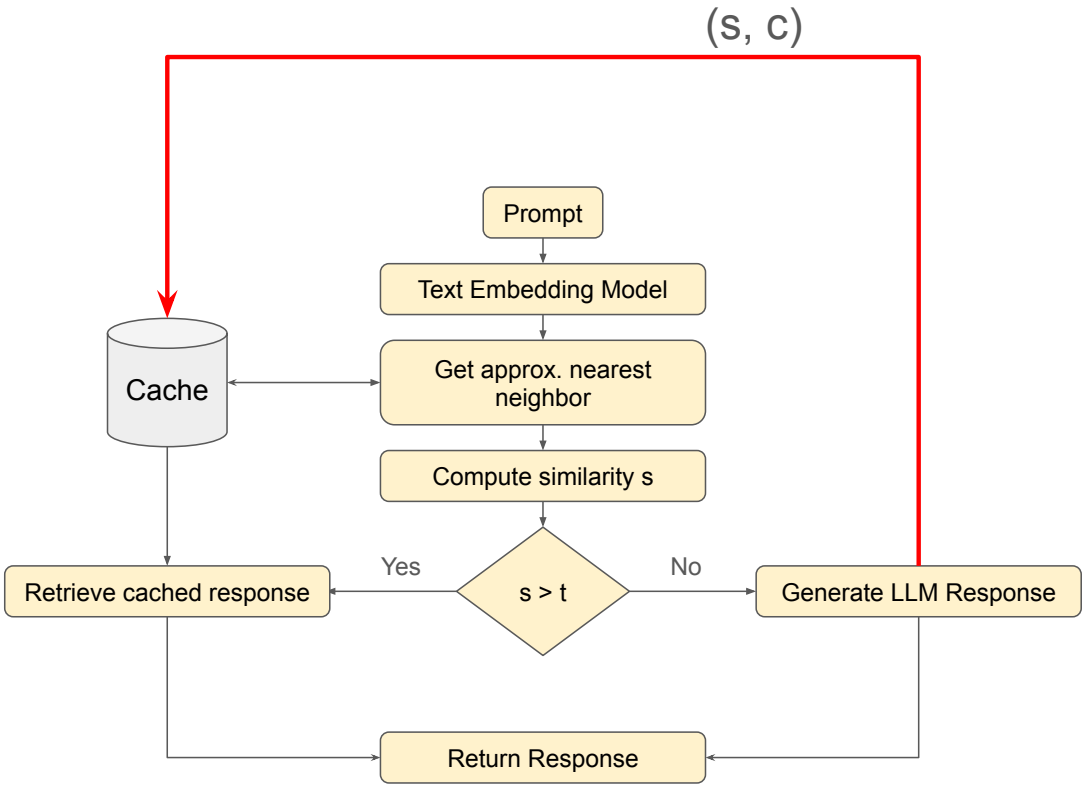


Full dataset

Sigmoid models it well

$$c = \frac{1}{1 + e^{-\gamma(s-t)}}$$

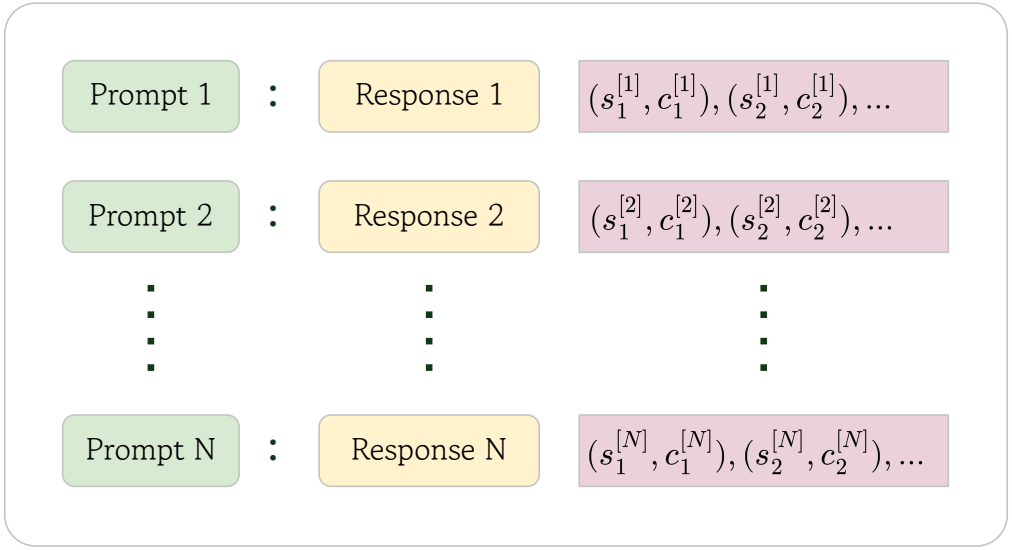
How to estimate this correctness from data?



Simple!

When you generate a new response, you get a datapoint (s, c) for the matched prompt

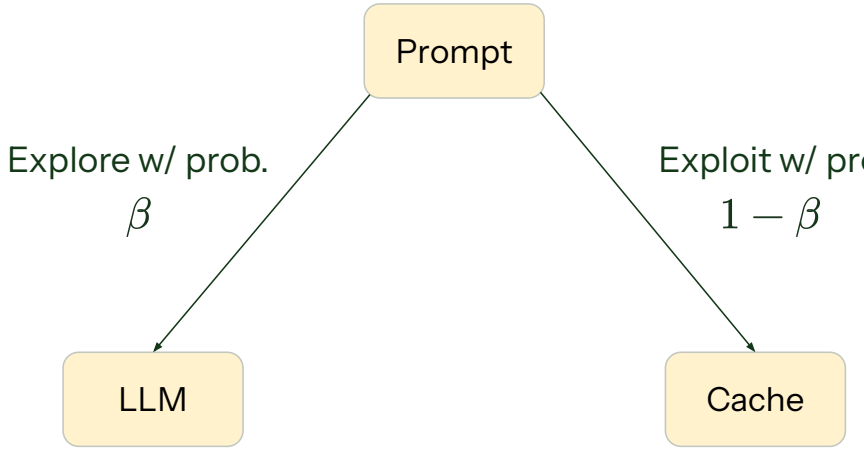
Cache Structure



Estimate the correctness using optimization with Binary Cross Entropy (BCE)

$$(\hat{t}, \hat{\gamma}) = \arg\min_{t,\gamma} \sum_{(s,c)\in O} \mathrm{BCE}\left(c, \hat{c}(s; t, \gamma)\right)$$

How to use this correctness estimate?



$$\alpha = \hat{c}(s; t, \gamma)$$

Error Guarantee

$$\mathbb{P}(correct) \geq 1 - \delta$$

$$\beta + (1-\beta)\alpha \geq 1 - \delta$$

$$\implies \beta \geq \frac{1-\delta-\alpha}{1-\alpha}$$

Confidence Bounds for Parameters

Exploration Threshold $\beta \geq \dfrac{1-\delta-\alpha}{1-\alpha}$

We need to ϵ-bound the probability of bad event

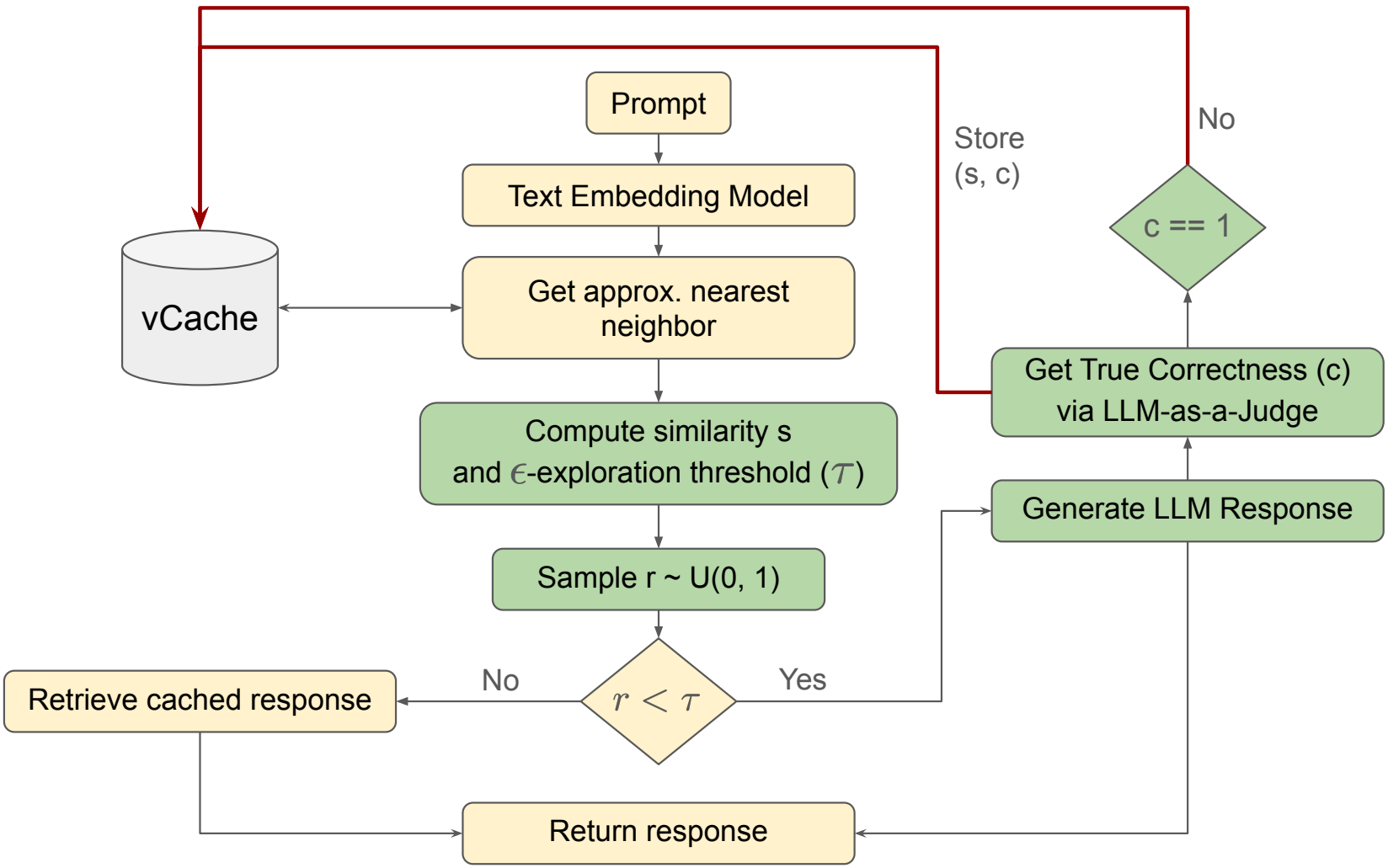
$$\mathbb{P}((t^* > t') \vee (\gamma^* < \gamma')) < \epsilon$$

which leads to the following result $\alpha \geq (1-\epsilon)\hat{c}(s; t', \gamma')$

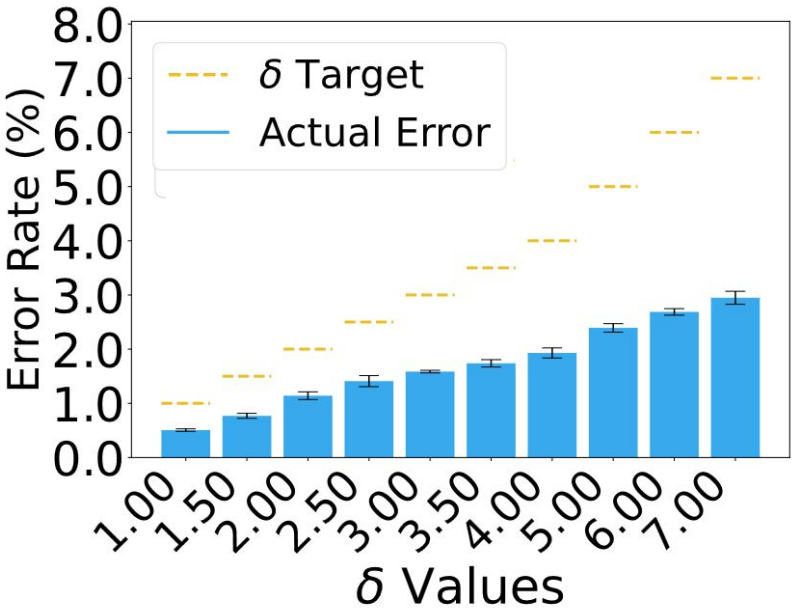
$$t', \gamma' \leftarrow \phi^{-1}(\hat{t}, \hat{\gamma}, 1-\epsilon)$$

ϵ-Exploration Threshold $\tau = \min_{\epsilon \in (0,1)} \dfrac{1-\delta-(1-\epsilon)\hat{c}(s; t', \gamma')}{1-(1-\epsilon)\hat{c}(s; t', \gamma')}$

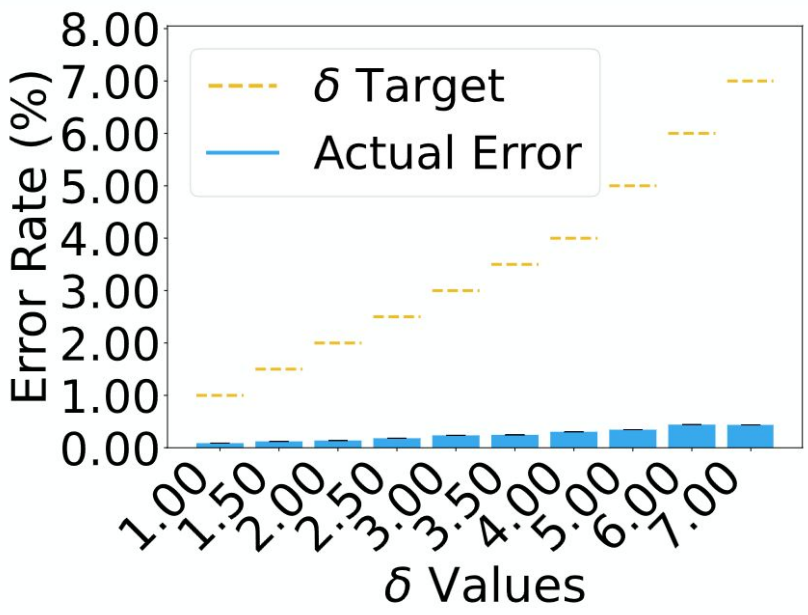
vCache Algorithm



Result: Error Rate is bounded

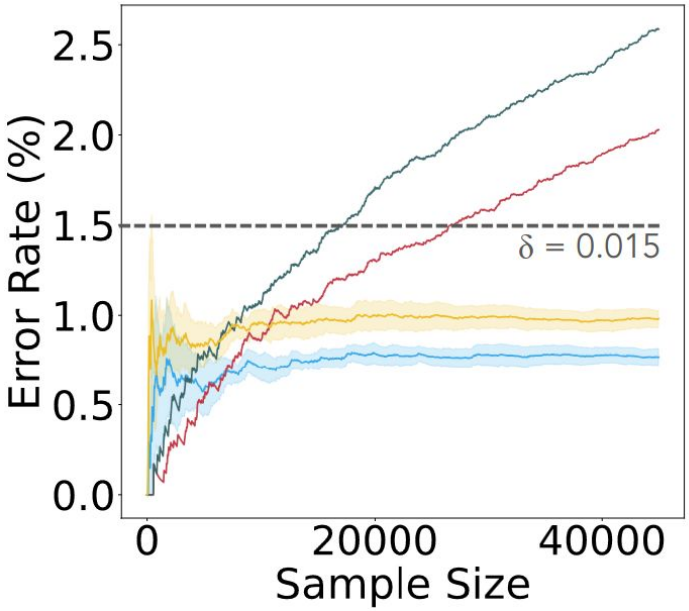


GteLargeENv1 embedding
Llama 3.1-8B LLM
SemCacheClassification bench

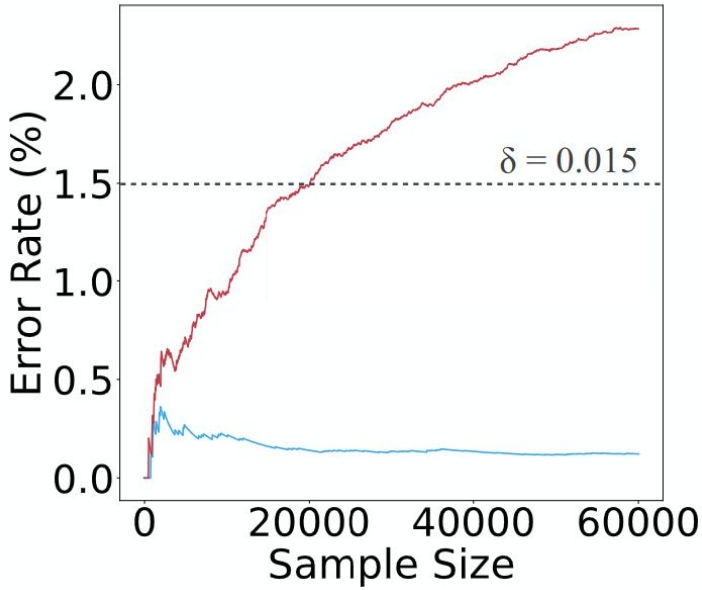


OpenAI text-embedding-3-small
OpenAI GPT 4.1-nano LLM
SemCacheLMArena bench

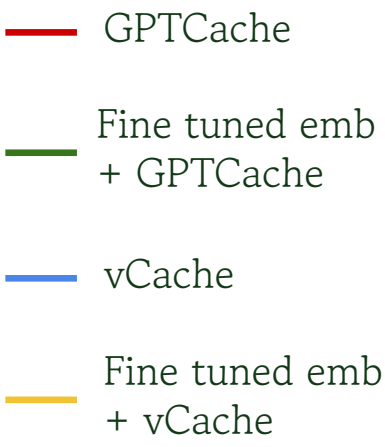
Result: Error Rate is bounded asymptotically



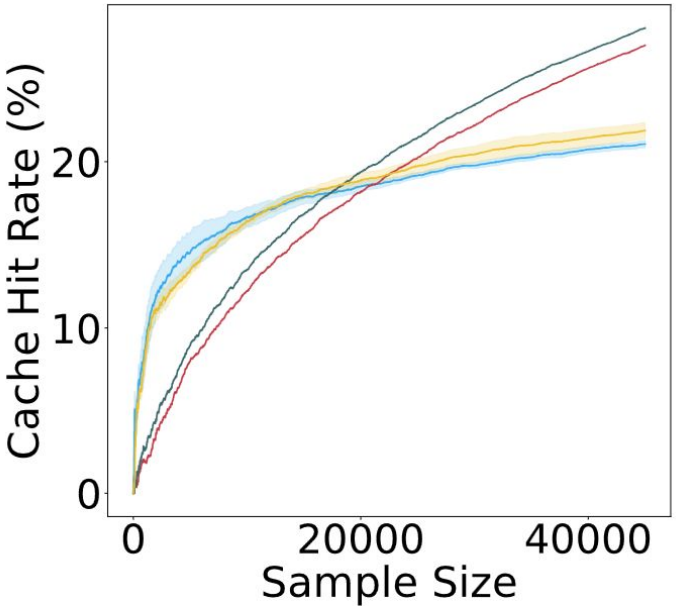
GteLargeENv1 embedding
Llama 3.1-8B LLM
SemCacheClassification bench



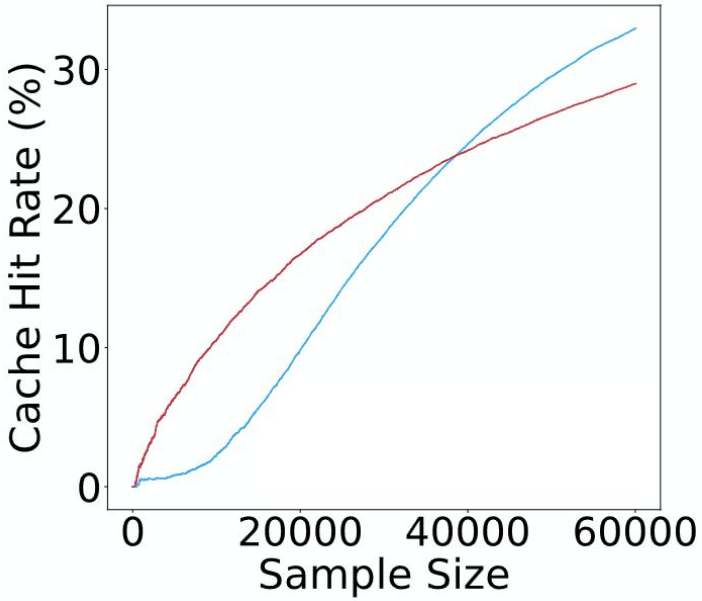
OpenAI text-embedding-3-small
OpenAI GPT 4.1-nano LLM
SemCacheLMArena bench



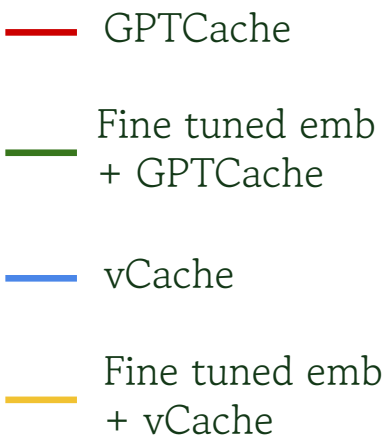
Result: Cache Hits remain high



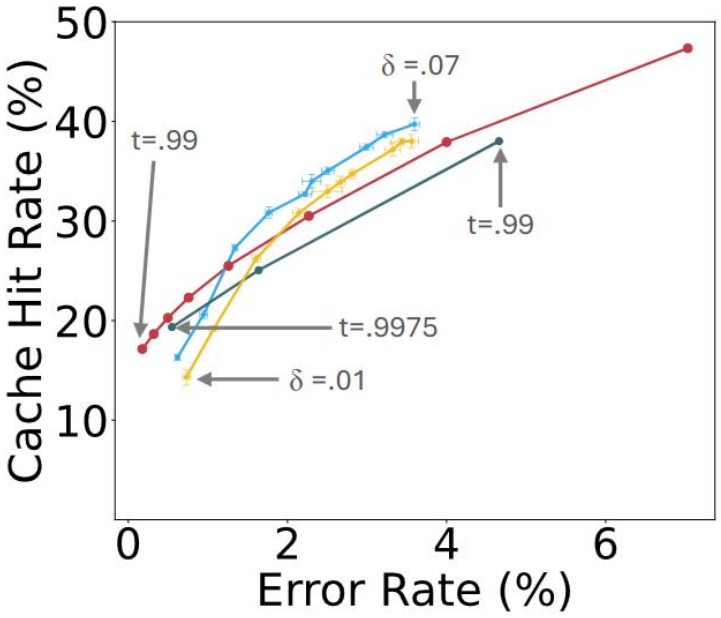
GteLargeENv1 embedding
Llama 3.1-8B LLM
SemCacheClassification bench



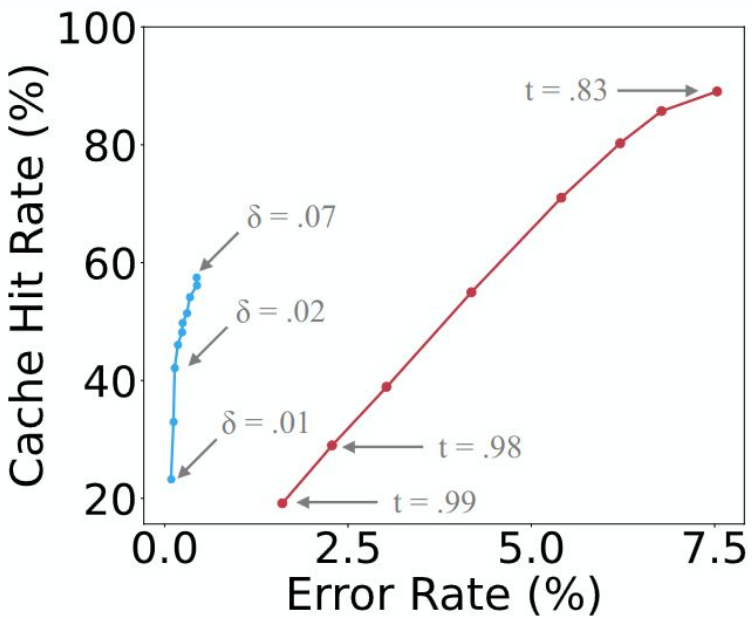
OpenAI text-embedding-3-small
OpenAI GPT 4.1-nano LLM
SemCacheLMArena bench



Result: Higher Cache Hits at same error rate



GteLargeENv1 embedding
Llama 3.1-8B LLM
SemCacheClassification bench



OpenAI text-embedding-3-small
OpenAI GPT 4.1-nano LLM
SemCacheLMArena bench

GPTCache

Fine tuned emb + GPTCache

vCache

Fine tuned emb + vCache

RouteLLM

learning to route LLMs with preference Data

Aadeshveer Singh

November 6, 2026

Problem Statement

LLMs come in all shapes and sizes, with different abilities. Models can be:

1. **Strong**: Big models with high accuracy but high costs
2. **Weak**: Small models with low accuracy but low costs

How to decide the best model for a given query?

Possible solutions

1. Train a median model with good enough cost and accuracy.(Expensive, may outdate in future and many other reasons)
2. Let the user decide(ad hoc, will likely be suboptimal)
3. **RouteLLM**: Bootstrap ML, use ML methods to choose which LLM gives output.

What sort of framework can we use?

Given a dataset of queries and user preference(strong model vs weak model[1]) we need a model that predicts the probability that strong model is more preferred. Let this probability be $P(q)$ for some query q. So a router is defined as

$$R^{\alpha}(q) = \begin{cases} \mathcal{M}_{weak} & \text{if } P(q) < \alpha \\ \mathcal{M}_{strong} & \text{else} \end{cases}$$

- $\mathcal{M}_{weak}$ is the weak/cheaper model and $\mathcal{M}_{strong}$ is the strong/expensive model.
- $\alpha \in [0, 1]$ is a cost threshold that defines our tradeoff between quality and cost.

[1] ML models are stochastic so a weak model may be preferred for some cases

Metrics

For standardization:

- Cost($c(R^\alpha)$) is defined as fraction of queries routed to strong model, i.e. cost of weak model is assumed as 0 and strong model as 1
- Performance($r(R^\alpha)$) is derived according to score function based on testing database(like a rating system, correctness etc)
- Performance is standardized to a 0 to 1 range as *Performance gap recovered (PGR)*

$$PGR(R^\alpha) = \frac{r(R^\alpha) - r(\text{weak model})}{r(\text{strong model}) - r(\text{weak model})}$$

- This allows us to make a pareto for cost to performance and the average performance is measured as area under it

$$APGR(R^\alpha) = \int_0^1 PGR(R^\alpha) d(c(R^\alpha))$$

- Finally we might not care about the average performance and so a *call-performance threshold(CPT)* is defined as given a router what is minimum cost to achieve given performance

Routing Approaches(1/4): Similarity-weighted (SW) ranking

Bradley–Terry model fit *at inference time*, with training queries weighted by similarity to the new query q.

- Embed queries (ϵ for q, ϵ_s for training query q_s) and compute a normalized cosine similarity

$$S(q, q_s) = \frac{\cos(\epsilon, \epsilon_s)}{\max_{s'} \cos(\epsilon, \epsilon_{s'})}$$

  so $S = 1$ for the closest training query
- Weight per training query: $\omega_s = \gamma^{1+S(q,q_s)}$, with $\gamma = 10$
- Learn BT coefficients ξ_s, ξ_w for strong/weak:

$$\arg\min_{\xi_s, \xi_w} \sum_{(q_s, l_{s,w}) \in D_{pref}} \omega_s \cdot \ell(l_{s,w},\ \sigma(\xi_s - \xi_w))$$

  with ℓ = binary cross-entropy
- Win probability: $P(q) = \sigma(\xi_s - \xi_w)$
- No training; refit per query (slowest to serve)

Paper's Eq. 9 takes the max over q''s neighbours; the released code normalizes per incoming query, as above.

Routing Approaches(2/4): Matrix Factorization

Idea from recommender systems: learn a hidden score $\delta(M, q) \in \mathbb{R}$ for how good model M is on query q.

- If M beats M' on q, then $\delta(M, q) > \delta(M', q)$. Enforced via

$$P(q) = \sigma\big(\delta(M_s, q) - \delta(M_w, q)\big)$$

- Bilinear score: $\delta(M, q) = w_2^T \left(v_m \odot (W_1^T v_q + b)\right)$
  - $v_q \in \mathbb{R}^{1536}$: frozen query embedding (text-embedding-3-small)
  - $W_1 \in \mathbb{R}^{1536 \times d_m}$ (+ bias b in the paper): projects the query into model space
  - $v_m \in \mathbb{R}^{d_m}$: one **learned** row per model in a $64 \times d_m$ lookup table ($d_m = 128$ in the code), not derived from model weights
  - $w_2 \in \mathbb{R}^{d_m}$: maps the product to a scalar
- Trained by gradient descent on arena battles: both models' rows, W_1, w_2 are updated to predict the winner
- Equivalent to a low-rank factorization of the model $\times$ query score matrix
- Adam, lr 3×10^{-4}, batch 64, $\sim$10 epochs on one 8GB GPU

Routing Approaches(3,4/4): BERT and Causal LLM

BERT

- Use normal BERT text classifier as base
- Win probability is predicted by sigmoid of

$$P(q) = \sigma(Wh_{CLS} + b)$$

- Here h_{CLS} is the embedding of query. and W, b are learn parameters

Causal LLM

- Llama 3 8B outputs the probability in next-token fashion
- Comparison labels are added as new tokens to the vocab; win probability is a softmax over the label tokens
- Needs fine-tuning, and is as big as many weak models, so costly to serve

Results: Setup

- $\mathcal{M}_{strong}$ = GPT-4 (1106-preview), $\mathcal{M}_{weak}$ = Mixtral-8x7B
- Training data: Chatbot Arena battles ($\sim$65k comparisons after pruning), models clustered into tiers to reduce label sparsity
- Augmentation:
  - D_{gold}: MMLU validation split ($\sim$1.5k), labels from comparing answers to the golden label
  - D_{judge}: $\sim$120k Nectar queries labeled by a GPT-4 judge ($\sim$\$700)
- Evaluation (out-of-domain): MT Bench, MMLU, GSM8K, contamination-checked
- Baseline: random router at the same cost

Results: MT Bench

Training data	Method	CPT(50%)	CPT(80%)	APGR (vs random)
	Random	49.03%	78.08%	0.500 (+0%)
Arena	BERT	78.09%	87.64%	0.391 (-21.8%)
	Causal LLM	28.82%	77.53%	0.573 (+14.6%)
	Matrix Factorization	25.32%	74.26%	0.580 (+16%)
	SW Ranking	37.85%	58.99%	0.610 (+22.1%)
Arena + D_{judge}	BERT	19.58%	34.02%	0.751 (+50.2%)
	Causal LLM	31.50%	48.75%	0.679 (+35.8%)
	Matrix Factorization	**13.40%**	**31.31%**	**0.802 (+60.4%)**
	SW Ranking	23.21%	36.04%	0.759 (+51.8%)

- High-capacity models (BERT, causal LLM) $\approx$ random on Arena alone: low-data regime
- CPT(50%) score is 8.8, i.e. 95% of GPT-4 (9.3)

Results: MMLU (5-shot)

Training data	Method	CPT(50%)	CPT(80%)	APGR (vs random)
	Random	50.07%	79.93%	0.500 (+0%)
Arena	BERT	49.43%	77.80%	0.502 (+0.5%)
	Causal LLM	48.88%	77.93%	0.499 (-0.2%)
	Matrix Factorization	45.00%	76.86%	0.524 (+4.9%)
	SW Ranking	55.82%	80.25%	0.473 (-5.4%)
Arena + D_{gold}	BERT	41.30%	72.20%	0.572 (+14.4%)
	Causal LLM	35.49%	70.31%	0.600 (+19.9%)
	Matrix Factorization	35.46%	71.40%	0.597 (+19.5%)
	SW Ranking	**35.40%**	71.55%	**0.603 (+20.7%)**

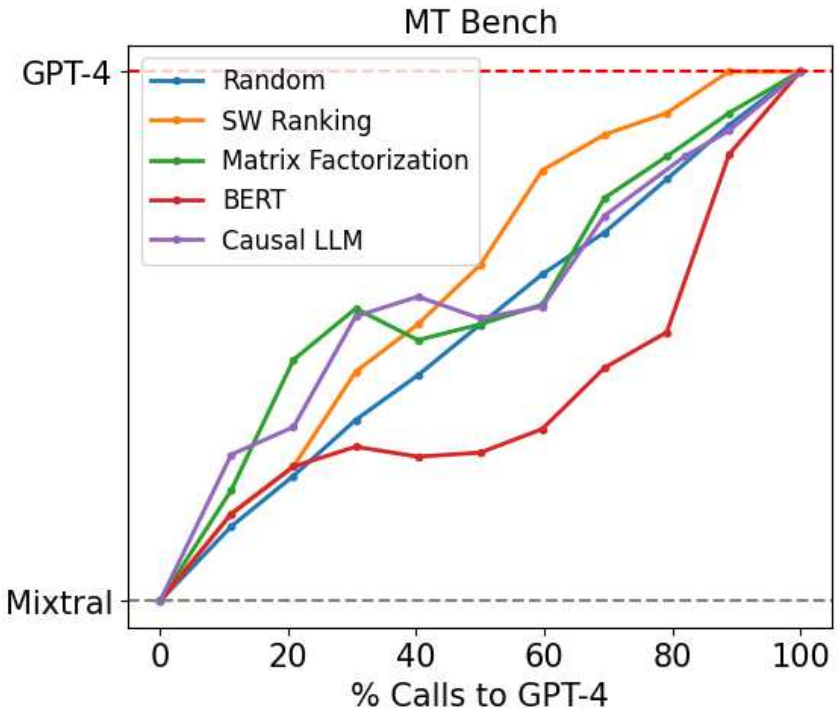
- Arena-only routers $\approx$ random: MMLU is mostly out-of-distribution
- Only $\sim$1.5k gold samples ($<$2% of data) give $\sim$20% fewer GPT-4 calls at CPT(50%)

Results: GSM8K (8-shot)

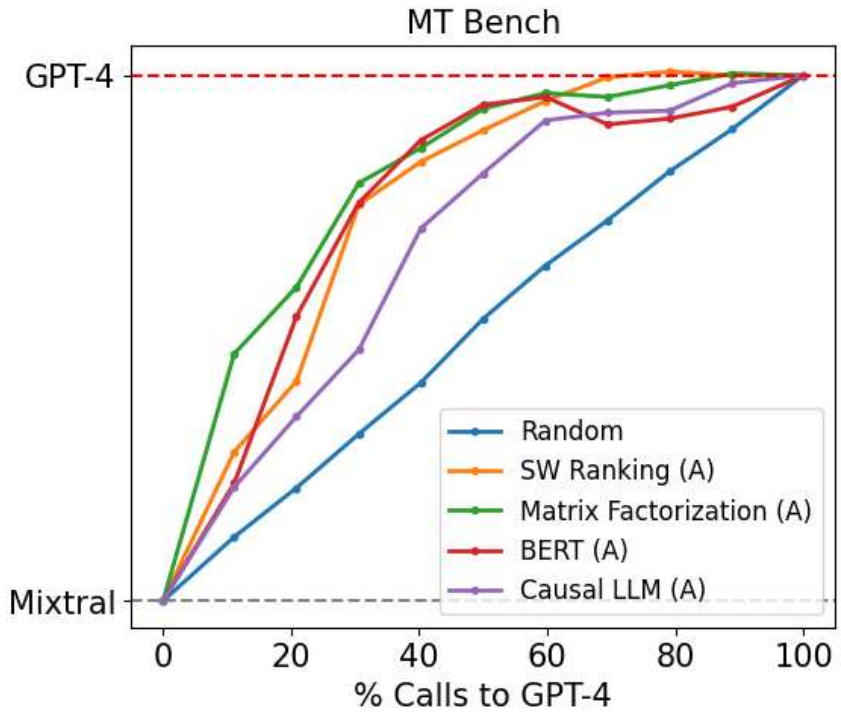
Training data	Method	CPT(50%)	CPT(80%)	APGR (vs random)
	Random	50.00%	80.08%	0.497 (+0%)
Arena	BERT	58.78%	83.84%	0.438 (-11.8%)
	Causal LLM	56.09%	83.56%	0.461 (-7.3%)
	Matrix Factorization	53.59%	85.24%	0.4746 (-4.5%)
	SW Ranking	54.43%	82.11%	0.4753 (-4.3%)
Arena + D_{judge}	BERT	44.76%	79.09%	0.531 (+6.9%)
	Causal LLM	**33.64%**	**63.26%**	**0.622 (+25.3%)**
	Matrix Factorization	38.82%	72.62%	0.565 (+13.8%)
	SW Ranking	41.21%	72.20%	0.568 (+14.3%)

- Arena-only routers are worse than random; augmentation flips all of them above random
- Causal LLM is best here (17% fewer GPT-4 calls than random)

Effect of data augmentation (MT Bench)



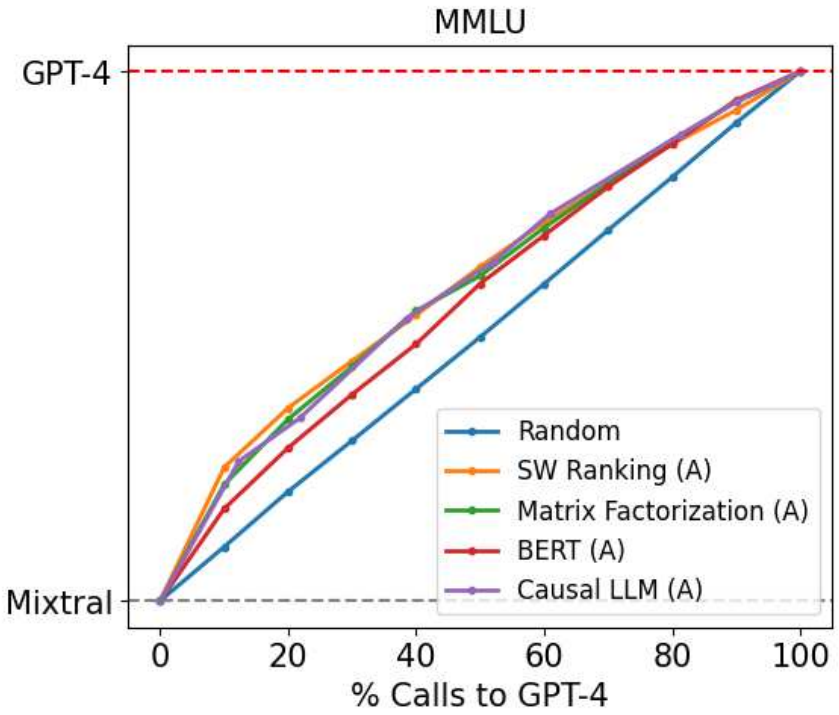
Arena only



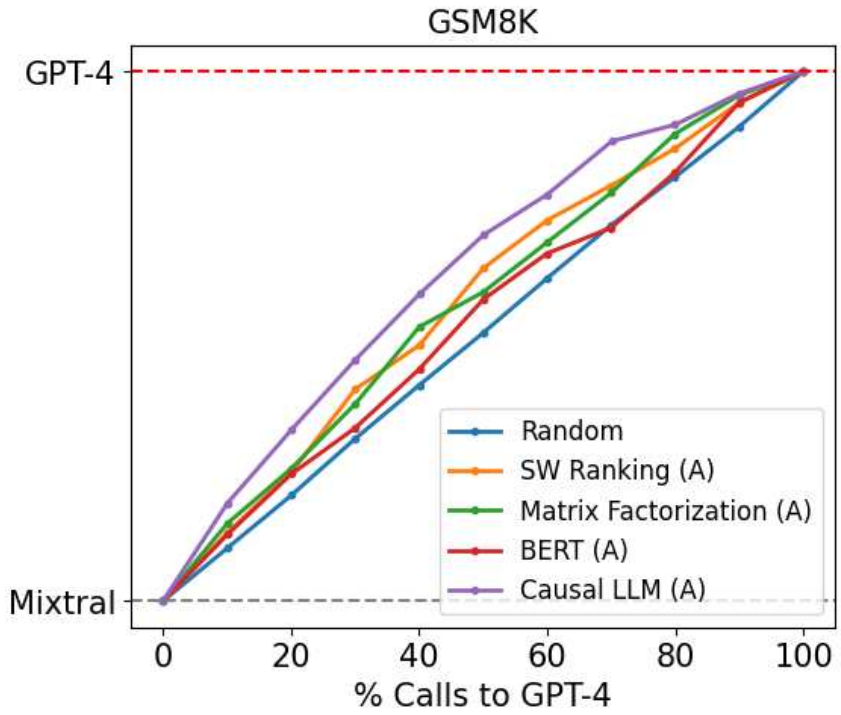
Arena + D_{judge} (A)

- Every router moves well above the random baseline after augmentation

Router curves on MMLU and GSM8K (augmented)



MMLU (Arena + D_{gold})



GSM8K (Arena + D_{judge})

- Gains are smaller than on MT Bench; they depend on how close the training data is to the benchmark (higher benchmark–dataset similarity $\Rightarrow$ better router)

Generalization to unseen model pairs (MT Bench)

Same routers, **no retraining**; only the strong/weak models being routed to are swapped.

Model pair (M_s / M_w)	Method	CPT(50%)	CPT(80%)	APGR (vs random)
Claude 3 Opus / Sonnet	Random	49.89%	72.27%	0.493
	BERT	34.85%	39.04%	0.682 (+38.3%)
	Causal LLM	28.12%	50.00%	0.656 (+33.1%)
	Matrix Factorization	31.86%	36.43%	0.762 (+54.6%)
	SW Ranking	23.27%	51.85%	0.772 (+56.6%)
Llama 3.1 70B / 8B	Random	47.52%	76.26%	0.512
	BERT	30.15%	38.91%	0.673 (+31.4%)
	Causal LLM	34.05%	45.96%	0.689 (+34.6%)
	Matrix Factorization	25.83%	37.30%	0.738 (+44.1%)
	SW Ranking	21.18%	29.39%	0.767 (+49.8%)

- The router has no vector for these models: it still scores the query with its GPT-4 / Mixtral knowledge and uses that as a proxy

Cost savings and routing overhead

Benchmark	CPT(50%) saving	CPT(80%) saving
MT Bench	3.66× (95% of GPT-4 quality)	2.49×
MMLU	1.41× (92%)	1.14×
GSM8K	1.49× (87%)	1.27×

Router	Cost / M requests	Requests / s
SW Ranking	\$39.26	2.9
Matrix Factorization	\$3.32	155.16
BERT	\$3.19	69.62
Causal LLM	\$5.23	42.46

- Savings = inverse of the router's GPT-4 call ratio vs random
- Even SW ranking (slowest) adds $\leq$0.4% over GPT-4 generation cost
- Cost of million request for small queries tested on is around \$9,000

Critique (1/2): throughput and latency

- **Throughput**: SW ranking serves 2.9 req/s per CPU VM (vs 155 for MF, 70 for BERT on one GPU)
  - Scales out, but $\sim$35 VMs at 100 req/s; it solves a weighted fit over the whole training set per query
- **Latency is never measured**: the paper argues against cascades for latency, then reports only req/s and dollars
  - Throughput $\neq$ per-request delay; the router sits on the critical path before the first token
- **Context and multi run**: Entire paper focuses on a single query(average 95 tokens), with no discussion on long context.

Critique (2/2): evaluation and reproducibility

- **Small, noisy benchmark**: MT Bench has 160 prompts; gains on MMLU/GSM8K are far smaller
- **Router variance unexplained**: the authors admit the best router changes by benchmark with no clear reason
- **Paper vs code**: SW normalization (Eq. 9) and BERT head (Eq. 13) do not match the released implementation

Thank you

Questions

SMART: Automatically Scaling Down Language Models with Accuracy Guarantees for Reduced Processing Fees

Paper by Saehan Jo and Immanuel Trummer (Cornell University)

arXiv:2403.13835

Presented by Vedant Saini

6th Nov 2026

Outline

Motivation

Method

Evaluation

Discussion

Bigger models, much bigger bills

- Providers sell a ladder of models at very different prices
- In the paper's lineup: \$0.03 vs. \$0.0004 per 1K input tokens — a **75**× spread
- Without ground-truth labels, users can't tell which cheap model is good enough
- So they default to the most expensive one

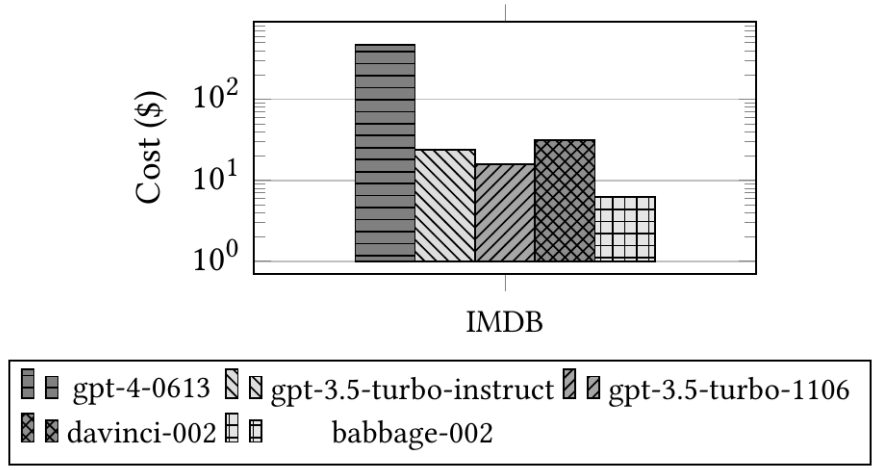


Fig. 1 of the paper: cost of each OpenAI model on IMDB sentiment (log scale).

Key idea: measure agreement, not correctness

- Treat the strongest LLM as the **reference** $\bar{m}$ (here GPT-4)
- A cheaper model is "accurate" on an item if its output **matches the reference's**
- Each item is a Bernoulli trial (match / no match) $\Rightarrow$ binomial statistics, no labels needed

Definition ($\langle \delta, \gamma \rangle$-equivalence)

An LLM is $\langle \delta, \gamma \rangle$-equivalent to $\bar{m}$ if its outputs are identical to those of $\bar{m}$ with probability at least $1 - \delta$, at confidence level γ.

Example

$\delta = 0.05,\ \gamma = 0.95$: at least 95% agreement with GPT-4, with 95% confidence.

Overview: two phases, three variants

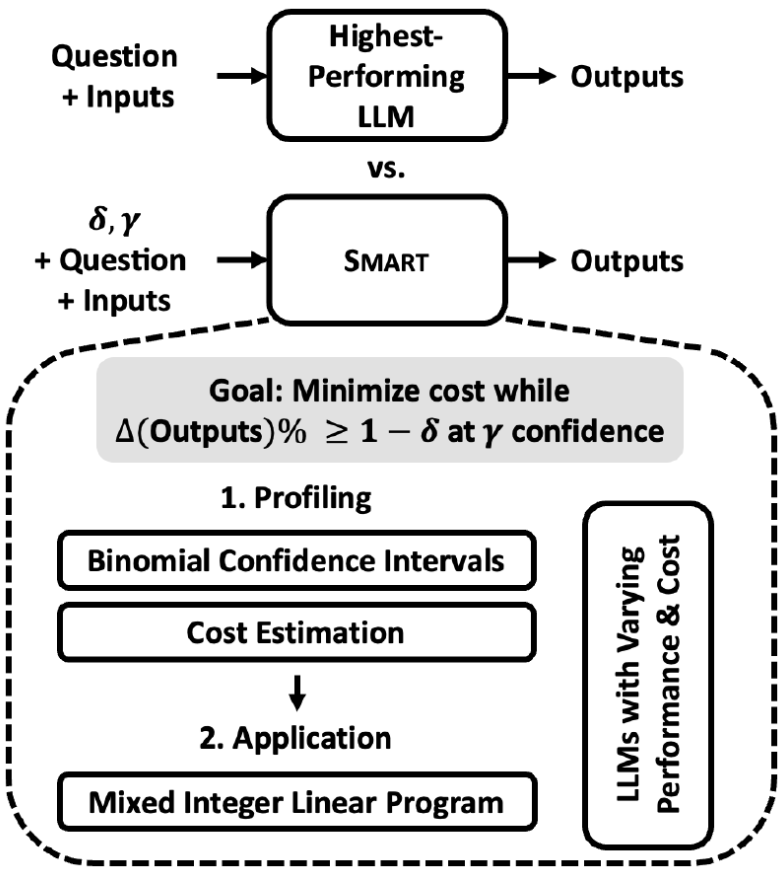


Fig. 2 of the paper.

Phases

1. **Profiling**: run $\bar{m}$ and every undecided cheap model on the same items; count agreement
2. **Application**: process the remaining items with cheap model(s) that meet the bound

Variants (each adds one feature)

- PROFILEALL: profile until the cheapest valid model is found
- PROFILESMART: + stop profiling when it stops paying off
- MODELMIX: + split work across several models (MILP)

ProfileAll: confidence intervals decide

- For each cheap model m: $m.n$ items seen, $m.e$ agreements
- After each item: Clopper–Pearson interval $[l, u]$ at level γ
  - $l \geq 1 - \delta \Rightarrow$ **Valid** (safe to use)
  - $u < 1 - \delta \Rightarrow$ **Invalid** (stop profiling it)
  - otherwise **Unknown** (keep profiling)
- Stop when the cheapest Valid model costs no more than any Unknown one

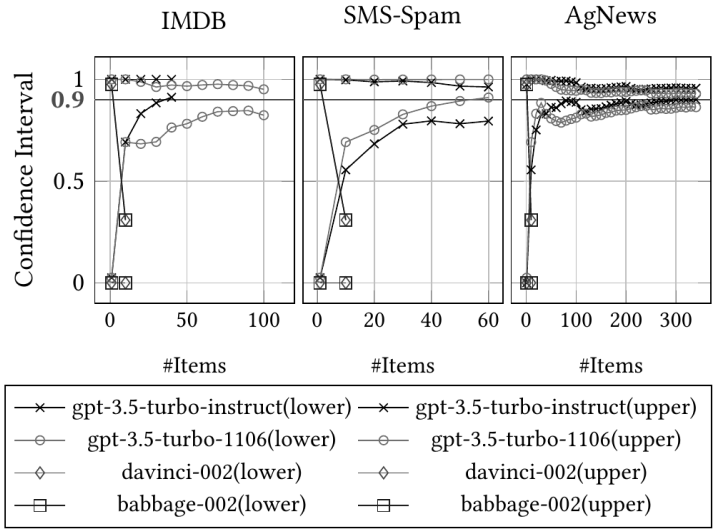


Fig. 8 of the paper: interval bounds during profiling, threshold $1 - \delta = 0.9$.

ProfileSmart: is more profiling worth paying for?

- Each profiled item costs $\bar{m}$ *plus* every Unknown model
- A model whose true agreement sits at $1 - \delta$ may never resolve
- Expected cost of profiling exactly k more items, then applying:

$$\begin{aligned}
\mathrm{Cost}(k) = k\Big(\bar{m}.c + \sum_i m_i.c\Big) & \qquad \text{profiling} \\
+ (n-k)\sum_i \Big(\prod_{j<i}(1-p_j)\Big)\, p_i\, m_i.c & \qquad \text{application} \\
+ (n-k)\prod_i (1-p_i)\, \bar{m}.c & \qquad \text{fallback}
\end{aligned}$$

- $p_i = P(m_i \text{ turns Valid}) = P(\mathrm{Binom}(k, a) \geq e^*)$, averaged over $a \sim \mathcal{N}\big(\hat{a}, \hat{a}(1-\hat{a})/m.n\big)$
- Try $k = 1, 2, 4, \cdots \leq n$; **stop if $k = 0$ is cheapest**

ProfileSmart: the cost trade-off in practice

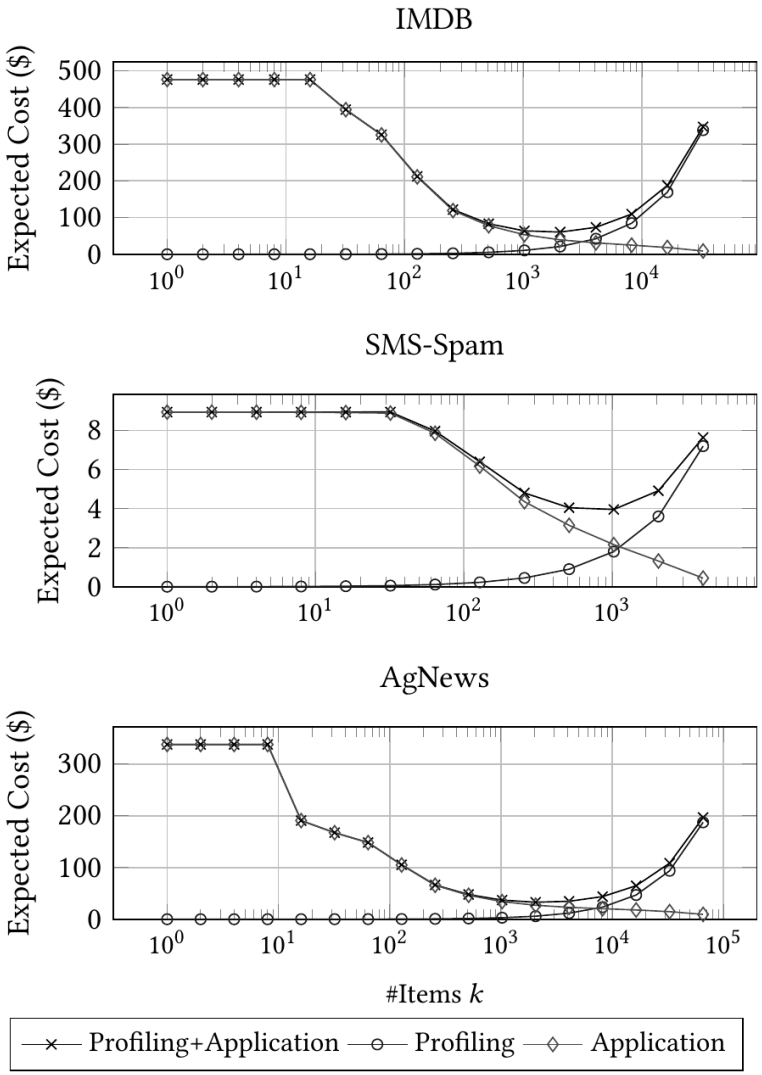


Fig. 9 of the paper: expected cost of profiling k more items after 100 profiled items, $\delta = 0.1$.

ModelMix: blend a near-miss with a stronger model

- A cheap model with lower bound just below $1 - \delta$ is rejected by the first two variants
- But agreement is additive over items: a **mix** can still clear the bar

Illustrative example (numbers not from the paper)

Target 0.90; cheap model's lower bound 0.86 at $\frac{1}{20}$ of GPT-4's price.

$$0.86\,x + 1.00\,(1 - x) \geq 0.90 \;\Rightarrow\; x \leq 0.71$$

$$\text{cost} \approx 0.71 \times 0.05 + 0.29 \times 1 \approx 0.32 \text{ of GPT-4} \;\; (\approx 3\times \text{ cheaper})$$

ModelMix: choosing the split with a MILP

$$
\begin{aligned}
\min \quad & \sum_i c_i x_i \\
\text{s.t.} \quad & \sum_{i,j} l_{ij}\, x_i\, y_{ij} \;\geq\; 1 - \frac{\delta}{1-r} && \text{(accuracy)} \\
& \sum_{i,j} y_{ij} \ln \gamma_j \;\geq\; \ln \gamma && \text{(confidence)} \\
& \sum_i x_i = 1, \qquad \sum_j y_{ij} \leq 1 \;\; \forall i \\
& x_i \in [0,1], \quad y_{ij} \in \{0,1\}
\end{aligned}
$$

- x_i: share of remaining items for model i; c_i: its unit cost
- y_{ij}: model i runs at confidence $\gamma_j \in \{\gamma, \gamma + 0.01, \ldots, 1\}$
- l_{ij}: interval lower bound of model i at γ_j; r: share answered by $\bar{m}$ during profiling
- Confidence: $\prod_i \gamma_i \geq \gamma$, linearised by taking logs

Experimental setup

Dataset	Task	Instances	Tokens/inst.
IMDB	Sentiment (2 labels)	50,000	293.7
SMS-Spam	Spam detection (2)	5,574	22.9
AgNews	News topic (4)	127,600	51.2

- **Models**: gpt-4-0613 (reference), gpt-3.5-turbo-instruct, gpt-3.5-turbo-1106, davinci-002, babbage-002
- **Settings**: $\gamma = 0.95$; $\delta \in \{0.02, 0.04, \ldots, 0.20\}$; 10 runs each
- **Metric**: cost savings = cost(GPT-4) / cost(SMART)
- **Caveat**: *simulated* — agreement rates measured on 100 real items per dataset, then full-size runs generated from those rates

Results: overall cost

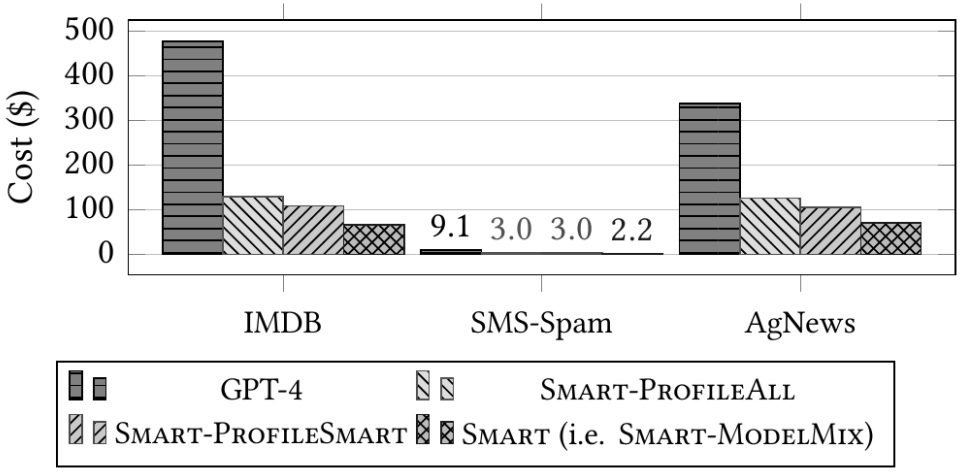


Fig. 3 of the paper: average cost over all δ.

Average savings of SMART vs. GPT-4:

- IMDB: **7.2**×
- SMS-Spam: **4.2**×
- AgNews: **4.8**×

MODELMIX over PROFILESMART: a further **1.4–1.6**×

Results: savings grow as δ relaxes

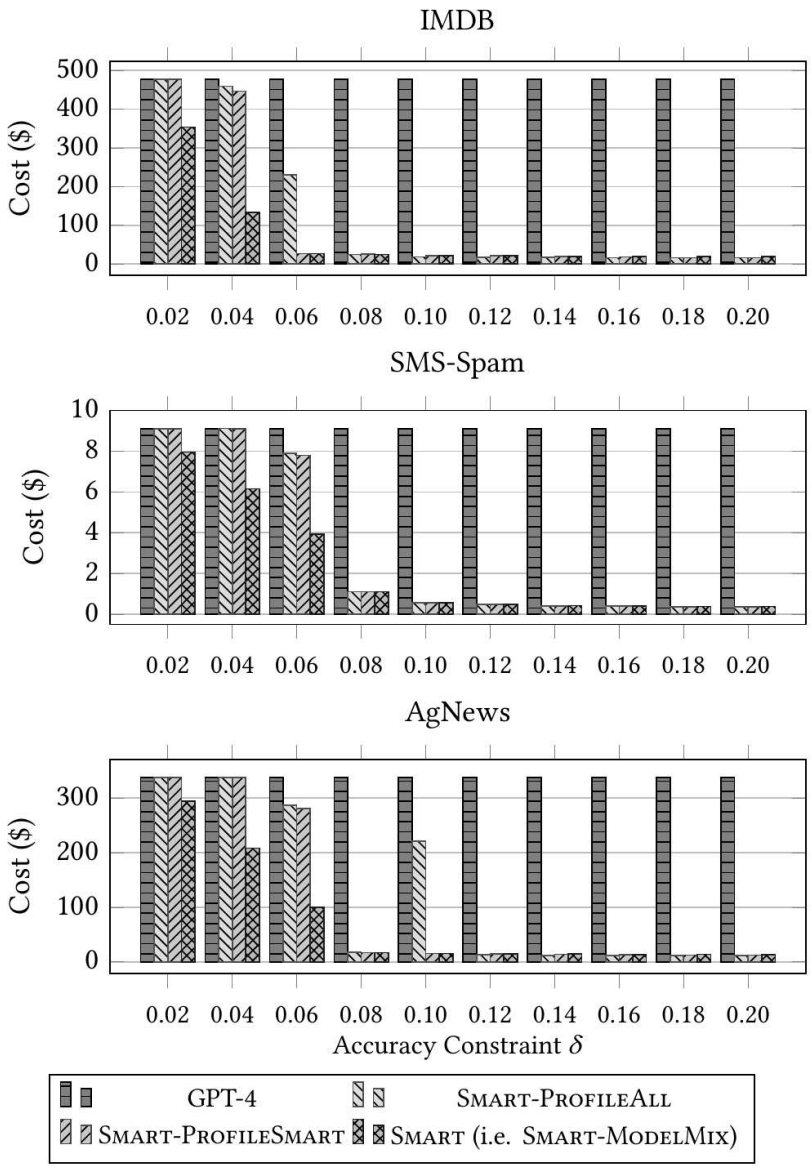


Fig. 4 of the paper.

Savings vs. GPT-4 (IMDB / SMS / AgNews):

- $\delta = 0.02$: 1.3× / 1.1× / 1.1×
- $\delta = 0.06$: 18.0× / 2.3× / 3.4×
- $\delta = 0.10$: 21.7× / 16.0× / 21.8×

At $\delta = 0.10$, GPT-4 processes only 0.2% / 2.6% / 0.2% of items.

Results: the guarantee holds

	IMDB			SMS-Spam			AgNews		
$1-\delta$	Min	Avg	#F	Min	Avg	#F	Min	Avg	#F
0.98	98.4	98.8	-	98.6	99.1	-	98.8	99.1	-
0.96	96.6	96.9	-	96.8	97.9	-	96.8	97.6	-
0.94	95.7	96.0	-	94.0	95.6	1/10	94.2	95.3	-
0.92	95.1	95.8	-	93.2	94.0	-	93.7	93.9	-
0.90	93.9	94.8	-	92.8	93.5	-	88.7	93.3	1/10
0.88	93.4	94.8	-	87.5	92.5	1/10	89.5	92.3	-
0.86	91.3	93.2	-	85.5	92.5	1/10	87.4	91.8	-
0.84	89.2	91.0	-	83.5	90.9	1/10	85.4	89.7	-
0.82	92.6	93.7	-	86.6	91.4	-	88.6	91.2	-
0.80	90.3	91.5	-	84.5	89.8	-	86.5	89.1	-

Table 3 of the paper: min/avg agreement (%) over 10 runs; #F = failed runs.

- Only **5 of 300** runs miss the target
- $\approx$1.7% failures, within the 5% allowed by $\gamma = 0.95$
- Where a run fails, only 1 in 10 does

Results: robustness to model accuracy

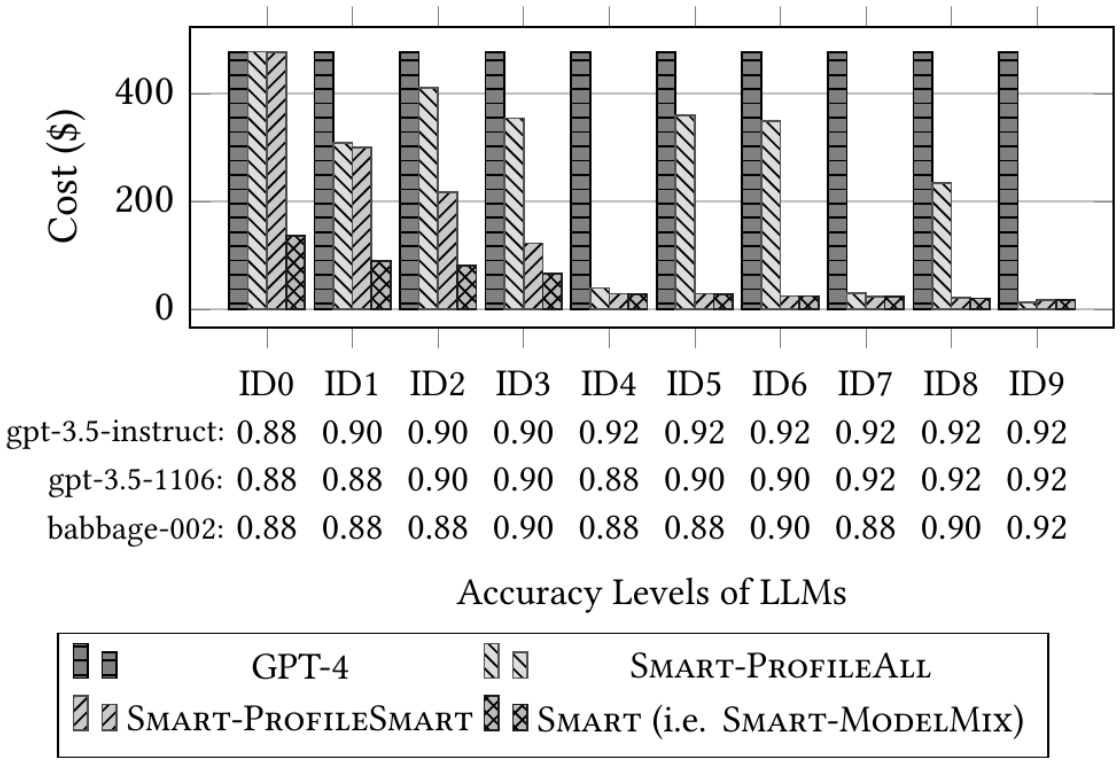


Fig. 7 of the paper: synthetic IMDB-like benchmark, $\delta = 0.1$; cheap models set to 0.88 / 0.90 / 0.92 agreement.

- All cheap models below threshold (ID0): only MODELMIX saves anything
- Models exactly at threshold: PROFILEALL wastes money on profiling

Questions worth raising

- **Agreement $\neq$ correctness**: GPT-4's mistakes are reproduced faithfully
- **Peeking after every item**: repeated interval checks across several models, with no correction for optional stopping or multiple comparisons
- **Simulated evaluation**: rates from 100 items, then i.i.d. runs — the test shares the method's assumptions
- **Not quite a MILP**: $l_{ij}\, x_i\, y_{ij}$ is bilinear; the linearisation is not described
- **Independence assumed**: $\prod_i \gamma_i \geq \gamma$ although all models see the same items
- **Classification only**: free-form outputs need an LLM judge — proposed, not tested

Takeaways

1. Agreement with a strong model is a usable, label-free quality signal
2. Profile only while profiling pays for itself
3. Mixing models beats picking one: 4–7× average savings, 16–22× at 90% agreement

Questions?

Code: `https://github.com/saehanjo/smart-llms` (Removed from repo, available on web archive)

Paper: `https://arxiv.org/abs/2403.13835`